

Claude + Power BI MCP

Detailed Configuration Guide for Power BI Report Generation

Goal Configure Claude Desktop to work with Power BI MCP servers for semantic-model authoring and report generation.

Target output A Power BI Project (PBIP) containing a TMDL semantic model, PBIR report definition, and theme JSON; then validate and save as PBIX.

Environment Windows 10/11, Claude Desktop, Power BI Desktop, Node.js, and optionally Git and VS Code.

Architecture at a glance

Component	Responsibility
Claude Desktop	Natural-language orchestration
Microsoft Power BI Modeling MCP	Semantic model, tables, relationships, measures
Power BI Report MCP (community)	Report pages, visuals, layout, filters, theme operations
PBIP project	TMDL + PBIR + theme resources
Power BI Desktop	Review, validate, save as PBIX

Important: MCP accelerates authoring, but generated reports still require review in Power BI Desktop and validation against trusted business results.

1. Understand the two MCP servers

Use separate MCP servers because semantic-model authoring and report authoring are different responsibilities.

Server	Purpose	Typical responsibilities
Microsoft Power BI Modeling MCP	Semantic model authoring	Create or modify tables, columns, relationships, measures, and model metadata.
Power BI Report MCP (community)	Report authoring	Create pages and visuals, configure visual properties, layout, filters, and theme-related settings.

Version note: exact tools, launch arguments, and supported operations can change. Follow each repository's current README. The Report MCP is a community project, not a Microsoft product.

2. Install prerequisites

Prerequisite	Purpose	Resource
Power BI Desktop	Open, inspect, validate, and save the project.	https://www.microsoft.com/en-us/download/details.aspx?id=58494
Claude Desktop	MCP client for local servers.	https://claude.ai/download
Node.js + npm	Run local MCP servers and build the report server.	https://nodejs.org/en/download
VS Code (recommended)	Edit configuration and inspect JSON/TMDL.	https://code.visualstudio.com/

Verify Node.js and npm in PowerShell:

```
node --version npm --version
```

If either command is not recognized, install Node.js, ensure it is on PATH, and reopen PowerShell.

3. Create a workspace

Keep the project, input data, and backups separate. Use one project folder per report.

```
mkdir C:\PowerBI-MCP mkdir C:\PowerBI-MCP\Projects mkdir C:\PowerBI-MCP\Data mkdir C:\PowerBI-MCP\Backups
C:\PowerBI-MCP\ ■■■ Projects\ ■ ■■■ Healthcare\ ■ ■■■ Healthcare.pbip ■■■ Data\ ■ ■■■ FactClaims.csv
■■■ Backups\
```

For an existing PBIX, open it in Power BI Desktop and use File → Save As → Power BI Project (.pbip), if supported by your Desktop version. Keep the original PBIX as a backup.

4. Install the MCP servers

4.1 Microsoft Power BI Modeling MCP

The official Microsoft Modeling MCP provides tools for working with Power BI semantic models. See [Microsoft Power BI Modeling MCP repository](#).

A package invocation may look like the following, but confirm the current README for the correct startup arguments and prerequisites:

```
npx -y @microsoft/powerbi-modeling-mcp@latest --start
```

Run the command manually first if you need to diagnose startup issues. Do not assume the sample command is valid for every released version.

4.2 Power BI Report MCP (community)

Clone and build the community server. Confirm the repository, branch, and setup instructions: [Power BI Report MCP repository](#).

```
cd C:\PowerBI-MCP git clone https://github.com/jonathan-pap/powerbi-report-mcp.git cd powerbi-report-mcp
npm install npm run build
```

Verify the expected entry point exists:

```
C:\PowerBI-MCP\powerbi-report-mcp\dist\index.js
```

If the build output or installation folder differs, use the actual path in your configuration.

5. Configure Claude Desktop

In Claude Desktop, open Settings → Developer → Edit Config. Alternatively, open the configuration file in PowerShell:

```
notepad "$env:APPDATA\Claude\claude_desktop_config.json"
```

Merge these entries into the existing mcpServers object. Do not replace other servers you already use.

```
{ "mcpServers": { "powerbi-modeling": { "command": "npx", "args": [ "-y",
"@microsoft/powerbi-modeling-mcp@latest", "--start" ] }, "powerbi-report": { "command": "node", "args": [
"C:\\PowerBI-MCP\\powerbi-report-mcp\\dist\\index.js" ] } } }
```

Version-sensitive: The Modeling MCP command is an example. If it fails, use the exact command documented for the installed version. The Report MCP may require additional environment variables or configuration.

6. Restart and verify the connections

- Save the configuration and completely quit Claude Desktop, including its background process.
- Start Claude Desktop again and open a new conversation.
- Open the tools / MCP interface and confirm the expected server tools are available.
- If tools are missing, inspect server logs and validate JSON syntax, paths, and executable availability.

Suggested read-only smoke tests

```
List the available Power BI MCP tools. Explain which tools can create semantic models and which can create report visuals.
```

7. Prepare a Power BI Project (PBIP)

Use PBIP as the working format because it exposes report and semantic-model definitions as project files. TMDL is used for the semantic model; PBIR is used for the report definition.

- Open the PBIP file in Power BI Desktop.
- Confirm the project opens without errors.
- Keep a clean backup or use source control before MCP modifications.
- Provide Claude the exact project path and ask it to inspect before changing anything.

Example inspection prompt

```
Connect to the Power BI project at: C:\PowerBI-MCP\Projects\Healthcare List the available tables, columns, measures, report pages, and visuals. Do not make any changes yet.
```

8. Test semantic-model generation

Start with inspection, then make one small, reviewable change. Do not ask Claude to generate the entire model before verifying its understanding of the schema.

Inspection prompt

```
Inspect the current semantic model. Identify: - Fact tables - Dimension tables - Relationships - Existing measures - Missing or ambiguous business definitions Do not modify the model yet.
```

Controlled measure-generation prompt

```
Create a measure named Total Claims Cost. Use the existing claims fact table and the appropriate paid amount column. Explain the DAX expression and list any assumptions before applying it.
```

Review the measure and validate it against an independent SQL query or an approved expected value. Check filter context and aggregation behavior, not just whether the expression compiles.

9. Test report-page generation

Once the model is verified, ask the Report MCP to create a small page using existing model fields and measures.

Example report-generation prompt

```
Create a report page named Healthcare Overview. Add: - A card for Total Claims Cost - A card for Claim Count - A line chart for monthly claims cost - A bar chart for claims cost by provider - A date slicer Use existing measures and columns from the model. Use a consistent healthcare theme. Do not invent fields or measures.
```

After generation, ask Claude to list the created visuals and verify each visual's field bindings, title, and page placement.

10. Generate and apply a theme

A theme JSON file can define report-wide colors, fonts, and supported formatting defaults. Keep theme generation separate from visual creation so the palette can be reused.

Example prompt

Apply a healthcare report theme with: - Navy blue as the primary color - Teal as the secondary color - White report background - Consistent typography - Clear visual titles - Accessible contrast Use the report theme feature. Do not overwrite an existing theme without showing me the proposed changes first.

Inspect the generated theme JSON and confirm it is valid and correctly referenced by the PBIR report. Individual visual formatting may override theme defaults.

11. Open and validate the generated report

- Save MCP-generated project changes.
- Open the .pbip file in Power BI Desktop.
- Refresh the report if needed and inspect rendering errors.
- Check missing fields, unsupported visuals, visual bindings, and layout.
- Validate DAX measures against SQL or trusted business results.
- Test slicers, filters, navigation, drill-through, and interactions.
- Save a separate .pbix copy after successful validation.

Validation checklist

Area	Validation question
Model	Are table grains, relationships, and filter directions correct?
DAX	Do key measures match independent calculations?
Visuals	Are the correct fields and measures assigned to each visual?
Filters	Do slicers and report/page/visual filters behave as intended?
UX	Are titles, labels, layout, contrast, and navigation clear?
Theme	Does the theme apply consistently without unwanted overrides?
Migration	Are unsupported or manually adjusted features documented?

12. Troubleshooting

Symptom	Checks
MCP tools do not appear	Restart Claude; verify config path and JSON; inspect logs.
npm is not recognized	Install Node.js, verify PATH, reopen PowerShell.
Server fails to start	Run its documented command manually; check version and dependencies.
Project not found	Verify the exact PBIP path and project access.
Visuals have missing fields	Compare visual bindings with actual model field names.
Theme does not apply	Validate JSON and confirm PBIR theme reference and visual overrides.
Desktop does not show updates	Save changes and reopen or refresh the project.
Model operation fails	Check whether the model is open, writable, and supported.

13. Recommended end-to-end workflow

Step	Activity	Output / gate
1. Describe	Provide business requirement, KPIs, pages, and interactions.	Clear scope and assumptions
2. Inspect	Claude inspects the model and existing report.	Confirmed schema and report inventory
3. Model	Modeling MCP creates or updates tables, relationships, and DAX.	Reviewed semantic model
4. Build	Report MCP creates pages, visuals, bindings, and filters.	Generated report pages
5. Style	Generate or apply theme and visual formatting.	Consistent visual language
6. Validate	Review calculations, bindings, layout, and interactions.	Validation evidence and issue list
7. Publish	Open in Desktop, save as PBIX, follow release process.	Validated deliverable

14. Recommended first MVP

Start with a simple healthcare claims report containing a source table, a few dimensions, two measures, one filter, and one chart. Automate only a narrow, testable slice before expanding.

- Parse one known Cognos or sample report specification.
- Extract basic report metadata and normalize it into an intermediate representation.
- Generate a small TMDL model with tables and supported measures.
- Generate a reusable theme JSON file.
- Generate one PBIR page containing a card, table, and bar chart.
- Produce a conversion report showing successful, review-required, and unsupported elements.
- Open and validate the PBIP in Power BI Desktop before saving as PBIX.

Recommended operating principle

Start with a small report and test each MCP independently. Combine the servers only after model and report operations work reliably. Keep backups, require approval before destructive changes, and treat generated DAX and visuals as unverified until reviewed.

Official and project references

- Microsoft Power BI Modeling MCP: <https://github.com/microsoft/powerbi-modeling-mcp>
- Power BI Report MCP (community): <https://github.com/jonathan-pap/powerbi-report-mcp>
- PBIR report structure: <https://learn.microsoft.com/en-us/power-bi/developer/projects/projects-report>
- TMDL authoring: <https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-tmdl-view>
- Claude Desktop download: <https://claude.ai/download>
- Claude local MCP setup: <https://support.claude.com/en/articles/10949351-getting-started-with-local-mcp-servers-on-claude-desktop>
- Power BI Desktop download: <https://www.microsoft.com/en-us/download/details.aspx?id=58494>

Note: This guide is a practical setup pattern. MCP capabilities, package names, command-line options, and configuration requirements are version-dependent. Verify current instructions in the linked repositories before installation.